Store Announcements API

Popups, top banners and menu cards on store websites — guide for frontend developers

Magskr backend · Base URL `https://magskr.com` · Version 1 · November 2026

What's inside

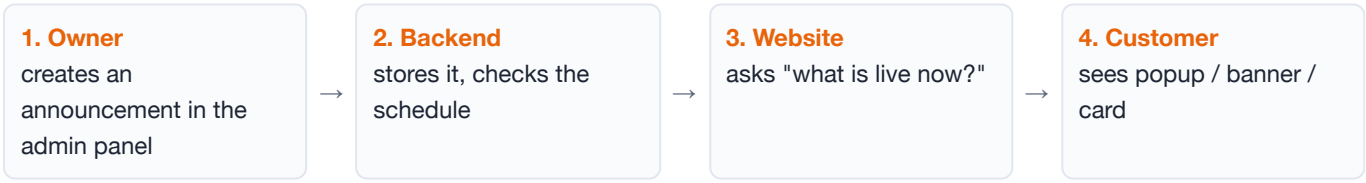
1. What this feature does
2. The 5 ideas you need to know
3. **Part A — Store website** (customer side): 5 steps
4. **Part B — Admin panel / owner app** (management side): 6 steps
5. Field reference
6. Errors and how to handle them
7. Launch checklist

1. What this feature does

Every restaurant can show messages on its ordering website, for example:

- A **popup** when the site opens: "*Christmas offer — 10% off with code XMAS10*"
- A thin **banner** at the top: "*Free delivery on orders over €30*"
- A **card inside the menu**: "*Lunch menu Mon–Fri 11:00–15:00 → see dishes*"

The restaurant owner creates them in the admin panel. The website fetches the ones that are live right now and shows them. Magskr (super-admin) can also create **platform-wide** notices, such as a maintenance message, which appear on **every** store's website.



2. The 5 ideas you need to know

① Placement — where it appears

placement	Where on the website	Suggested UI
`popup`	Modal on page load	Centered dialog, image on top, close (×) button
`banner`	Bar at the top of every page	One line of text + optional button, full width
`menu_card`	Inside the menu list	Card above the first category (or between categories)

② Language — German or English

Send `?lang=de` or `?lang=en`. The backend already picks the right text. If English text is missing, you get the German text. **You never need to choose between `title` and `title_en` yourself** on the website.

③ Schedule — when it is live

The backend decides this. The public API only returns announcements that are live **right now** (Berlin time). Owners can set any mix of: a date range (`start_at` / `end_at`), weekdays (`days_of_week`), and a daily time window (`daily_start_time` / `daily_end_time`). **The website does not need to check dates.**

④ Frequency — how often one customer sees it

This is the **one thing the website must do itself**, so customers are not annoyed by the same popup on every page.

frequency	Meaning	Where to remember "shown"
`always`	Show every time the page loads	Nowhere
`once_per_session`	Once until the browser tab is closed (default)	`sessionStorage`
`once_per_day`	Once per calendar day	`localStorage` with today's date
`once_ever`	Only once for this browser	`localStorage`

Use the `version` field in the storage key: `ann:{id}:{version}` . When the owner edits an announcement, its `version` changes, so customers see the updated message again.

⑤ Button (CTA) — what happens on click

cta_type	cta_value contains	What the website should do
`none`	—	No button
`url`	`https://…` or a path like `/menu`	Open the link (external links in a new tab)
`category`	Category id, e.g. `"412"`	Scroll to / open that menu category
`product`	Product id, e.g. `"9031"`	Open that product's detail / add-to-cart dialog
`coupon`	Coupon code, e.g. `"XMAS10"`	Copy the code to clipboard and show "Copied!" (or apply it to the cart)

Button text is in `cta_label` . If it is empty, use a default such as "Mehr erfahren" / "Learn more".

Part A — Store website (customer side)

No login needed. Five steps, in this order.

Step 1 Fetch live announcements when the site loads

`GET` `/store-announcements/public/{store_id}?lang=de`

Optional query: `placement=popup` | `banner` | `menu_card` to get only one type. Easiest is to fetch all once and split them in the frontend.

```
GET https://magskr.com/store-announcements/public/42?lang=en
```

Response — a list, **already sorted** (platform notices first, then by the owner's order). Empty list `[]` means "show nothing".

```
[
  {
    "id": 7,
    "is_platform": true,
    "placement": "banner",
    "title": "Maintenance tonight 02:00–03:00",
    "body": null,
    "image_url": null,
    "cta_type": "none",
    "cta_value": null,
    "cta_label": null,
    "dismissible": true,
    "frequency": "once_per_day",
    "display_order": 0,
    "version": "1762851600"
  },
  {
    "id": 15,
    "is_platform": false,
    "placement": "popup",
    "title": "Christmas offer",
    "body": "10% off all orders until 24 December.",
    "image_url": "https://magskrimages.s3.amazonaws.com/8f2c...e1.jpg",
    "cta_type": "coupon",
    "cta_value": "XMAS10",
    "cta_label": "Copy code",
    "dismissible": true,
    "frequency": "once_per_session",
    "display_order": 1,
    "version": "1762855200"
  }
]
```

Step 2 Remove the ones this customer has already seen

```
const today = () => new Date().toISOString().slice(0, 10);   // "2026-11-11"
const keyOf = a => `ann:${a.id}:${a.version}`;

function alreadySeen(a) {
  try {
    switch (a.frequency) {
      case "always":           return false;
      case "once_per_session": return sessionStorage.getItem(keyOf(a)) === "1";
      case "once_per_day":     return localStorage.getItem(keyOf(a)) === today();
      case "once_ever":        return localStorage.getItem(keyOf(a)) === "1";
    }
  } catch (e) { /* storage blocked (private mode) → just show it */ }
  return false;
}

function markSeen(a) {
  try {
    if (a.frequency === "once_per_session") sessionStorage.setItem(keyOf(a), "1");
    if (a.frequency === "once_per_day")     localStorage.setItem(keyOf(a), today());
    if (a.frequency === "once_ever")        localStorage.setItem(keyOf(a), "1");
  } catch (e) {}
}
```

Step 3 Show them in the right place

```
const res  = await fetch(`${API}/store-announcements/public/${storeId}?lang=${lang}`);
const list = (await res.json()).filter(a => !alreadySeen(a));

const banners = list.filter(a => a.placement === "banner");     // show all, stacked
const cards   = list.filter(a => a.placement === "menu_card");  // show all, in order
const popups  = list.filter(a => a.placement === "popup");

showBanners(banners);
showMenuCards(cards);
if (popups.length) showPopup(popups[0]);   // one at a time — next one after close
```

Popups: show **only one at a time**. When the customer closes it, show the next one (or wait for the next page load). Never stack several modals.

- **When to call `markSeen(a)`:** as soon as the item is actually displayed on screen.
- **Image:** `image_url` is a normal public link — use it directly in `<img>`. It can be `null`: then show text only.
- **Body:** plain text. Keep line breaks (CSS `white-space: pre-line`). Do not render it as HTML.

Step 4 Handle the close button (dismissible)

- `dismissible: true` → show a close (×) button. Clicking outside the popup may also close it.
- `dismissible: false` → no close button; for a popup, closing happens only through the CTA button. Use this for important notices (e.g. "We are closed today").

Step 5 Handle the button click (CTA)

```
function onCtaClick(a) {
  switch (a.cta_type) {
    case "url":      window.open(a.cta_value, a.cta_value.startsWith("/") ? "_self" :
"_blank"); break;
    case "category": scrollToCategory(Number(a.cta_value)); break;
    case "product":  openProduct(Number(a.cta_value));      break;
    case "coupon":   navigator.clipboard.writeText(a.cta_value); toast("Code copied!");
break;
  }
  closeAnnouncement(a);
}
```

Caching: the API is fast and cached. Fetch once per page load. You do not need polling. Changes made by the owner appear within seconds on the next page load.

Part B — Admin panel / owner app (management side)

All requests need the logged-in user's token:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Who is logged in	What they can do
Store owner	Only their own store. `store_id` can be left out — the backend uses their store.
Multi-store owner	Only the stores assigned to them. Always send `store_id`.
Super-admin	Any store, plus platform-wide notices (`store_id: null`).

Step 1 Show the list of announcements

GET `/store-announcements/?store_id=42` (optional `&placement=popup`)

Returns **all** announcements of the store — including inactive, future and expired ones — with every field (both languages), sorted by `display_order`. Super-admin without `store_id` gets the platform-wide list.

> Tip: show a status badge per row, calculated in the frontend: **Inactive** (`is_active` false), **Scheduled** (`start_at` in the future), **Expired** (`end_at` in the past), otherwise **Live**.

Step 2 Upload the image (optional)

POST `/images/upload` — multipart form, field name `file`

Use the returned `url` as `image_url` in the next step. The link returned here is temporary, but **the backend automatically converts it to the permanent link** when you save the announcement — so just pass it as-is. Recommended image: landscape, about 1200×675 px, JPG/WebP.

Step 3 Create an announcement

POST `/store-announcements/` → **201 Created**, returns the full row

Only `title` is required. Everything else has a sensible default.

```
{
  "store_id": 42,
  "placement": "popup",
  "title": "Weihnachtsangebot",
  "body": "10% Rabatt auf alle Bestellungen bis 24. Dezember.",
  "title_en": "Christmas offer",
  "body_en": "10% off all orders until 24 December.",
  "image_url": "https://magskrimages.s3.amazonaws.com/8f2c...e1.jpg",
  "cta_type": "coupon",
  "cta_value": "XMAS10",
  "cta_label": "Code kopieren",
  "cta_label_en": "Copy code",
  "frequency": "once_per_session",
```

```
  "dismissible": true,
  "is_active": true,
  "start_at": "2026-12-01T00:00:00",
  "end_at":   "2026-12-25T00:00:00"
}
```

Example — weekday lunch card (every Mon–Fri, 11:00 to 15:00, linking to a category):

```
{
  "placement": "menu_card",
  "title": "Mittagsmenü",
  "title_en": "Lunch menu",
  "days_of_week": [0, 1, 2, 3, 4],
  "daily_start_time": "11:00",
  "daily_end_time": "15:00",
  "cta_type": "category",
  "cta_value": "412",
  "cta_label": "Gerichte ansehen",
  "frequency": "always"
}
```

Form tips

- Dates: send local Berlin time without timezone (`"2026-12-01T00:00:00"`), or with a timezone — both work.
- Weekdays: `0` = Monday … `6` = Sunday. Leave empty (`null`) for every day.
- Daily times: fill both or neither. Overnight is allowed (22:00 → 02:00).
- For `category` / `product` buttons, let the owner pick from a dropdown and send the id as a string.

Step 4 Edit an announcement

`PUT` `/store-announcements/{id}` → returns the updated row

Send only the fields that changed. Fields you leave out stay as they are. To clear an optional field, send it as `null` .

```
PUT /store-announcements/15
{ "body": "20% Rabatt bis Sonntag!", "end_at": null }

PUT /store-announcements/15        ← switch off without deleting
{ "is_active": false }
```

`store_id` cannot be changed. Every edit gives the announcement a new `version` , so customers see it again.

Step 5 Change the order (drag & drop)

`PATCH` `/store-announcements/reorder` → returns the rows in the new order

After the owner drags rows, send all new positions in one call. Lower number = shown first.

```
{
  "store_id": 42,
  "items": [
    { "id": 15, "display_order": 0 },
    { "id": 18, "display_order": 1 },
    { "id": 16, "display_order": 2 }
  ]
}
```

Step 6 Delete an announcement

DELETE `/store-announcements/{id}` → **204 No Content** (empty body)

Deleting is permanent. Show a confirm dialog first, and suggest "Deactivate" (`is_active: false`) as the safer option.

5. Field reference

Field	Type / default	Description
`id`	int, read-only	Announcement id
`store_id`	int or null	The store. `null` = platform-wide (super-admin only). Cannot be changed after creating.
`placement`	`"popup"`	`popup` · `banner` · `menu_card`
`title`	string, **required**	German title, max 200 characters
`body`	string or null	German text (plain text, line breaks allowed)
`title_en` / `body_en`	string or null	English versions; fallback to German when empty
`image_url`	string or null	Public image link, max 500 characters
`cta_type`	`"none"`	`none` · `url` · `category` · `product` · `coupon`
`cta_value`	string or null	Required when `cta_type` is not `none`. URL / id / coupon code.
`cta_label` / `cta_label_en`	string or null	Button text, max 80 characters
`display_order`	int, `0`	Lower = shown first
`is_active`	bool, `true`	Master on/off switch
`dismissible`	bool, `true`	Show a close (×) button
`frequency`	`"once_per_session"`	`always` · `once_per_session` · `once_per_day` · `once_ever`
`start_at` / `end_at`	datetime or null	Live from `start_at` until (not including) `end_at`. Berlin time.
`days_of_week`	list of int or null	0 = Monday … 6 = Sunday; null = every day
`daily_start_time` / `daily_end_time`	`"HH:MM"` or null	Daily window; both or neither
`created_at` / `updated_at`	datetime, read-only	Timestamps (Berlin time)

Public API only: `is_platform` (true for Magskr-wide notices) and `version` (changes on every edit — use it in the "seen" key). The text fields there are already in the requested language.

6. Errors and how to handle them

Errors return JSON with a `detail` field. It comes in **two shapes** — handle both:

```
{ "detail": "end_at must be after start_at." }                    ← string: show it as-is

{ "detail": [ { "loc": ["body", "days_of_week", 0],               ← list: wrong field type/value
                "msg": "Value error, days_of_week values must be 0 (Monday) .. 6 (Sunday)"
} ] }
```

For the list shape, show each `msg` next to the field named in `loc` (last item). This happens for things like a missing title, an unknown `placement`, or a weekday outside 0–6.

Code	When	Example `detail`
400	Store id does not exist	Store with ID 999 does not exist.
401	Missing / expired token	Send the user to login
403	Not allowed for this store	You are not authorized for this store.
404	Wrong id, or id from another store	Announcement not found.
422	Invalid form values	end_at must be after start_at. · cta_value is required when cta_type is 'coupon'. · Set both daily_start_time and daily_end_time, or neither. · cta_value must be an http(s) URL or a site path starting with '/'.

On the **public website**, if the request fails for any reason, simply show no announcements — never block the ordering page.

7. Launch checklist

Store website

- ☐ Fetch `/public/{store_id}` once on page load, with the site's current language
- ☐ Filter by frequency using the `ann:{id}:{version}` key
- ☐ Banners at the top, menu cards in the menu, popups one at a time
- ☐ Close button only when `dismissible` is true
- ☐ All 4 CTA types work (url, category, product, coupon)
- ☐ Works with no image, no body and no button
- ☐ Body shown as plain text, not HTML
- ☐ API error → page still works, nothing shown
- ☐ Looks good on mobile (popup fits the screen, image scales)

Admin panel

- ☐ List with status badges (Live / Scheduled / Expired / Inactive)
- ☐ Create / edit form with German + English tabs and image upload
- ☐ Category / product picker for CTA buttons
- ☐ Date range, weekday and time window inputs
- ☐ Drag & drop reorder → one `PATCH /reorder` call
- ☐ Deactivate toggle and delete with confirm dialog
- ☐ A live preview of the popup / banner while editing (nice to have)